Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN

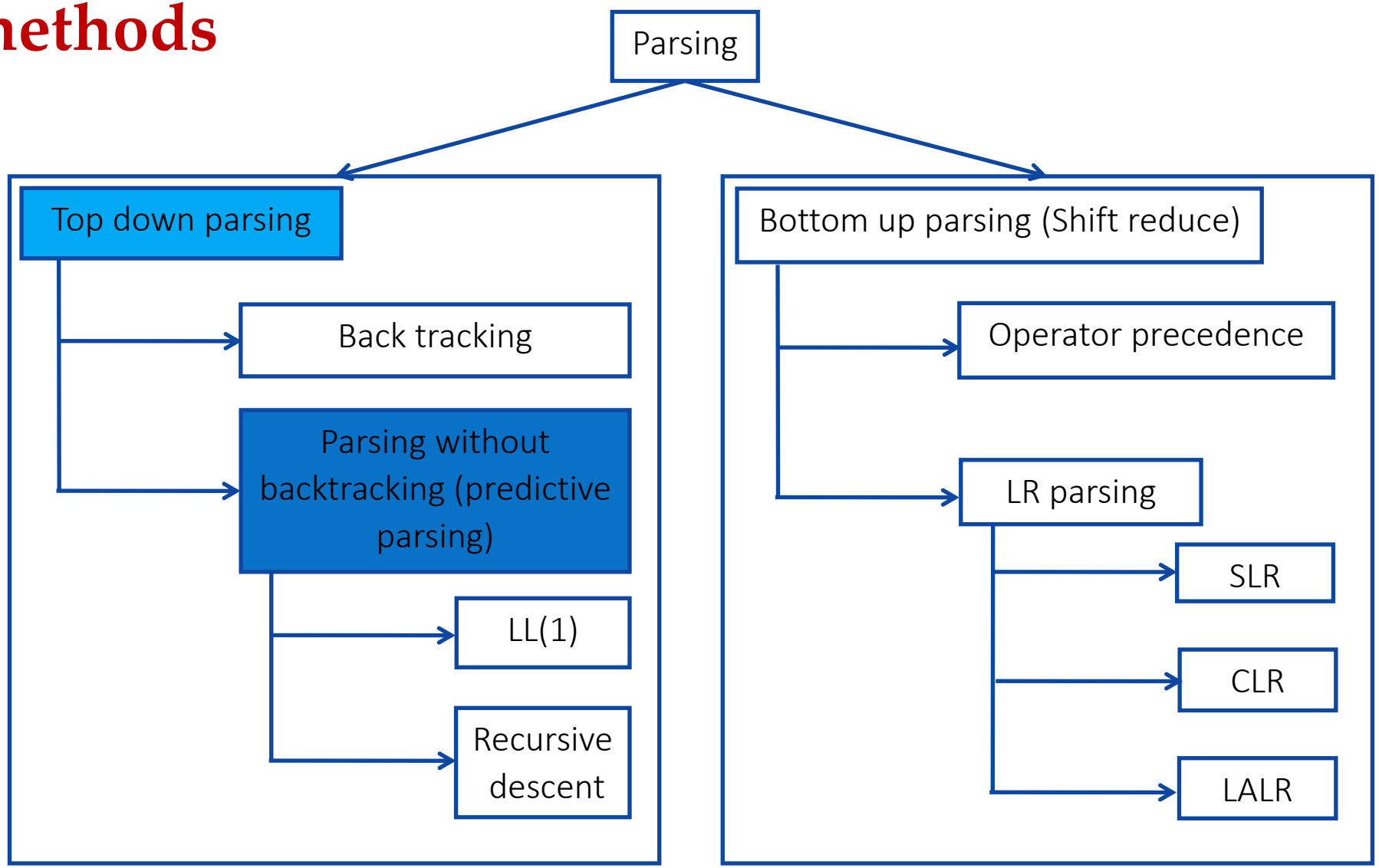
Module 2

SYNTAX ANALYSIS

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

RECURSIVE DESCENT PARSING

Parsing methods



- A top down parsing that executes a set of ***recursive procedure to process the input without backtracking*** is called recursive descent parser.
- The term **descent** refers to the ***direction in which the parse tree is built***.
  - It builds the parse tree from the root to the leaf.
- Write ***a procedure for each non terminal in the grammar.***
- ***Consider RHS of any production rule as definition of the procedure.***
- As it reads expected input symbol, it advances pointer to next input position.

```
void A() {
        Choose an A-production, A → X1X2 ··· Xk;
        for ( i = 1 to k ) {
                if ( Xi is a nonterminal )
                        call procedure Xi();
                else if ( Xi equals the current input symbol a )
                        advance the input to the next symbol;
                else /* an error has occurred */;
        }
}
```

- Consider the following grammar:

E -> iE^{l}

E^{l} -> +iE / ε

- E and E’ are non-terminals and ‘+’, ‘i’ are characters.
- Epsilon represents the end of the recursion

 # Procedure for E

```
E()
{
 # l is the lookahead
      if (l == ‘i’)
       {
              match(‘i’);
              E’();
    }
}
```

 # Procedure for E’

```
E’()
{
       if (l == ‘+’)
         {
              match(‘+’)
              match(‘i’)
               E’();
              }
else
      return;
}
```

 # Procedure to match a character and input new character

```
match(char t)
{
    if (l == t) {
              l = getchar();
    }
    else{
              printf("Error");
     }
}
main()
{
       E();
       if (l==’$’){
printf(“Parsing Successful”);
              }
}
```

Example: Recursive Descent parsing

```
{
        If lookahead=num
        {
                Match(num);
                T();
        }
        Else
                Error();
        If lookahead=$
        {
                Declare success;
        }
        Else
                Error();
}
```

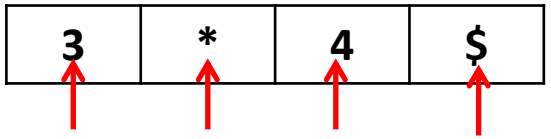
```
{
        If lookahead='*'
        {
                Match('*');
                If lookahead=num
                {
                        Match(num);
                        T();
                }
                Else
                        Error();

        }
        Else
                NULL
}
```

```
Proceduce Match(token t)
{
    If lookahead=t
    lookahead=next_token;
    Else
        Error();
}
```

```
Procedure Error
{
            Print("Error");
}
```

$E \rightarrow num\ T$

$T \rightarrow * num\ T \mid \epsilon$



Success

Example: Recursive Descent parsing

```
Procedure E
{
        If lookahead=num
        {
                Match(num);
                T();
        }
        Else
                Error();
        If lookahead=$
        {
                Declare success;
        }
        Else
                Error();
}
```

```
Procedure T
{
        If lookahead='*'
        {
                Match('*');
                If lookahead=num
                {
                        Match(num);
                        T();
                }
                Else
                        Error();

        }
        Else
                NULL
}
```

```
Proceduce Match(token t)
{
    If lookahead=t
    lookahead=next_token;
    Else
        Error();
}
```

```
Procedure Error
{
            Print("Error");
}
```

$E \rightarrow num\ T$

$T \rightarrow * num\ T \mid \epsilon$

3	*	4	$

Success

3	4	*	$

Error

Example 2 : Write down the algorithm using Recursive procedures to implement the following Grammar

$E \rightarrow TE'$
$E' \rightarrow +TE'$
$T \rightarrow FT'$
$T' \rightarrow * FT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

```
Procedure E
{
    T();
    E'();
}
```

```
Procedure T
{
    F();
    T'();
}
```

```
Procedure E'
{
 if lookahead == '+'
    match ('+');
 T();
 E'();
}
```

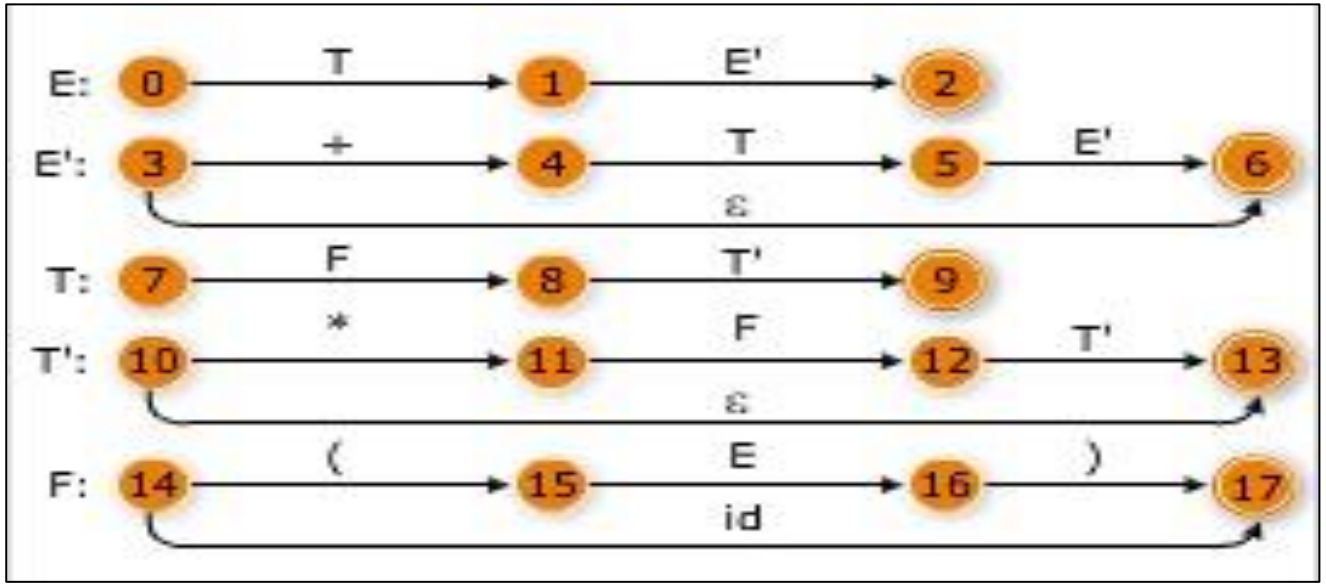
```
Procedure T'()
{
 if lookahead == '*'
    match ('*');
    f();
    T'();
 else
    NULL;
}
```

```
Procedure match(token x)
{
 if lookahead == x
    lookahead = next_token;
 else
    Error();
}
```

```
Procedure Error
 {
  printf("Error");
 }
```

```
Procedure f()
{
 if lookahead == 'id'
    match ('id');
 else if lookahead == 'c'
    match ('c');
  E();
 if lookahead == ')'
    match(')');
 else
    Error();
else
    Error()
 if lookahead == '$'
  Declare Success
else
  Error();
}
```

Transition diagrams



Code for parser

```
Procedure E()
Begin
T()
E'()
End
```

```
Procedure F()
Begin
if input_symbol = '+' then
      begin
                  ADVANCE()
                  T()
                  E'()
      end
```

```
Procedure T()
Begin
F()
T'()
End
```

```
Procedure T'()
Begin
if input_symbol = '*' then
      begin
                  ADVANCE()
                  F()
                  T'()
      end
```

```
Procedure F()
Begin
if input_symbol = 'id' then
    ADVANCE()
Else if input_symbol = '(' then
          begin
               ADVANCE()
                  E()
                if input_symbol = ')' then
                        ADVANCE()
                 else ERROR()
                 end
Else ERROR()
```